Worketyamo

Accélérateur de Talents Tech

COURS COMPLET

Formation Dev Web Full Stack

JavaScript



Du premier script à la maîtrise du langage

Prérequis : HTML · CSS · Git · Figma · Tailwind

Comment utiliser ces exercices ?

Ces exercices sont conçus pour que tu **apprennes en faisant**, pas en relisant le cours de A à Z.

Méthode recommandée :

1. Lis la question et l'indice associé.
2. Identifie dans le cours le ou les concepts dont tu as besoin (voir référence du chapitre indiquée).
3. Rédige ta réponse dans les lignes prévues ou directement dans ton éditeur.
4. Teste ton code dans la console du navigateur (F12) ou dans VS Code + Node.js.

★☆☆ **Facile** — Consolidation des bases | ★★☆ **Moyen** — Combinaison de concepts | ★★★ **Avancé** — Situation réelle complexe

EXERCICES — SITUATIONS PROBLÈMES

JavaScript

Chapitres 1 à 6 · Variables, Contrôle, Fonctions,

Tableaux, Objets, Strings

3

Exercices

6

Chapitres couverts

EXERCICE 01 ★☆☆

La Caisse Enregistreuse

Contexte : *Tu développes un petit système de caisse pour une boutique de téléphones à Yaoundé. Le système doit gérer les articles, calculer les totaux et afficher des récapitulatifs.*

📖 **Rappel de cours — Variables & Types (Chap. 1)**

- const pour les valeurs fixes, let pour les valeurs qui changent
- Les types : string, number, boolean, null, undefined
- typeof pour vérifier le type d'une valeur
- Conversion : Number(), String(), Boolean()

Question 1 — Les données des articles

💡 **Indice :** *Relis le Chap. 1 sur const/let et les types avant de répondre.*

a) Déclare 3 variables représentant un article : son nom (ex: `'iPhone 15'`), son prix en FCFA (ex: `450000`) et s'il est en stock (`true` ou `false`). Utilise `const` ou `let` selon si la valeur peut changer. Justifie ton choix.

b) Que retourne `typeof prix` ? Et `typeof estEnStock` ? Pourquoi `typeof null` retourne-t-il `'object'` et non `'null'` ?

c) L'utilisateur saisit le prix dans un champ HTML, tu reçois la valeur `'450000'` (une string). Écris le code pour convertir cette string en nombre et stocker le résultat dans une variable `prixNum`. Que vaut `Number('')` et `Number('abc')` ?

📖 **Rappel de cours — Tableaux (Chap. 4) — à lire avant Q2**

- Créer : const tab = [val1, val2, ...]
- Accéder : tab[0], tab.length, tab.at(-1)
- .push() pour ajouter, .pop() pour retirer le dernier
- .map() transforme chaque élément → nouveau tableau
- .filter() garde les éléments selon une condition
- .reduce() accumule → une seule valeur (ex: somme)

Question 2 — Gérer le panier

💡 **Indice :** *Relis les méthodes .map(), .filter() et .reduce() dans le Chap. 4 avant de répondre.*

Voici le tableau représentant le contenu du panier d'un client :

```
const panier = [  { id: 1, nom: 'iPhone 15',     prix: 450000, quantite: 1,
categorie: 'premium' },  { id: 2, nom: 'Câble USB-C',   prix: 5000,   quantite:
3, categorie: 'accessoire' },  { id: 3, nom: 'Coque silicone',prix: 8000,
quantite: 2, categorie: 'accessoire' },  { id: 4, nom: 'Samsung A54',   prix:
185000, quantite: 1, categorie: 'standard' },]
```

a) Utilise `.map()` pour créer un nouveau tableau `nomsProduits` contenant seulement les noms des articles.

b) Utilise `.filter()` pour obtenir uniquement les articles de catégorie `'accessoire'`. Stocke le résultat dans `accessoires`.

c) Utilise `.reduce()` pour calculer le montant total du panier (prix × quantité pour chaque article). La valeur initiale de l'accumulateur doit être `0`.

d) BONUS — Enchaîne `.filter()` et `.reduce()` en une seule expression pour calculer le total des accessoires uniquement.

📖 **Rappel de cours — Strings (Chap. 6) — à lire avant Q3**

– Template literals : `Texte ${variable}` avec backticks

- .toUpperCase(), .toLowerCase(), .trim()
- .padStart(n, '0') pour formater avec des zéros
- .includes() pour vérifier la présence d'un texte
- String(nombre) pour convertir un nombre en texte

Question 3 — Formater le ticket de caisse

💡 **Indice :** *Relis les template literals et les méthodes de strings du Chap. 6 avant de répondre.*

a) En utilisant un template literal (backticks `` ` ``), génère une ligne de ticket formatée pour un article. Le résultat attendu doit ressembler à :

```
iPhone 15          x1        450 000 FCFA
```

Utilise `.padEnd()` pour aligner les colonnes. Le montant doit être formaté avec des espaces ou avec `Intl.NumberFormat`.

b) Génère un numéro de ticket à 6 chiffres à partir d'un simple nombre (ex: `42` → `'TKT-000042'`). Utilise `String()` et `.padStart()`.

c) Écris une fonction `verifierEmail(email)` qui retourne `true` si la chaîne contient un `@` et un `'.'` après le `@`. Utilise `.includes()` et `.indexOf()`.

EXERCICE 02 ★★☆

Le Gestionnaire d'Étudiants

Contexte : *Tu construis un système de gestion des notes et présences pour une école. Le programme doit calculer des moyennes, classer les étudiants et générer des bulletins.*

📖 **Rappel de cours — Structures de contrôle (Chap. 2)**

- if / else if / else pour les conditions multiples
- L'opérateur ternaire : condition ? siVrai : siFaux
- Les valeurs falsy : false, 0, '', null, undefined, NaN
- for...of pour itérer sur un tableau
- break pour sortir d'une boucle, continue pour passer à l'itération suivante

Question 1 — Évaluer la mention d'un étudiant

💡 **Indice :** *Relis les structures if/else et l'opérateur ternaire dans le Chap. 2.*

a) Écris une fonction `getMention(moyenne)` qui retourne :

```
≥ 16 → 'Très Bien'  |  ≥ 14 → 'Bien'  |  ≥ 12 → 'Assez Bien'  |  ≥ 10 → 'Passable'  |  < 10 → 'Insuffisant'
```

b) Réécris la même logique en utilisant un seul opérateur ternaire imbriqué. Quelle version est plus lisible ? Justifie.

📖 **Rappel de cours — Fonctions (Chap. 3)**

- Arrow function : const f = (param) => expression
- Paramètres par défaut : function f(x, y = 0) {...}
- Valeur de retour avec return
- Portée : une variable const/let dans une fonction n'est pas accessible dehors
- Closures : une fonction interne peut accéder aux variables de sa fonction parente

Question 2 — Traiter la liste de la classe

💡 **Indice :** *Relis les fonctions (Chap. 3) et les méthodes de tableaux (Chap. 4).*

Voici la liste des étudiants :

```
const etudiants = [  { nom: 'Alice Mengue',  notes: [14, 16, 12, 18, 15], absences: 1 },  { nom: 'Bob Tamba',     notes: [8,  10, 7,  9,  11], absences: 4 },  { nom: 'Clara Nkono',   notes: [13, 15, 14, 16, 12], absences: 0 },  { nom: 'David Fouda',   notes: [9,  11, 8,  10, 12], absences: 6 },  { nom: 'Emma Biyong',   notes: [17, 18, 16, 19, 15], absences: 0 },]
```

a) Écris une arrow function `calculerMoyenne(notes)` qui prend un tableau de notes et retourne la moyenne (utilise `.reduce()`).

b) Utilise `.map()` pour créer un nouveau tableau `bulletins` où chaque étudiant est enrichi de sa `moyenne` (arrondie à 2 décimales) et de sa `mention`. N'oublie pas le spread `{...etudiant, moyenne, mention}`.

c) Utilise `.filter()` pour extraire les étudiants ayant plus de 3 absences ET une moyenne inférieure à 10. Utilise l'opérateur logique `&&`.

d) Trie le tableau `bulletins` par moyenne décroissante avec `.sort()`. Quelle est la différence entre `.sort()` (mutant) et `.toSorted()` (non mutant) ?

📖 **Rappel de cours — Objets (Chap. 5)**

- Créer : const obj = { cle: valeur, ... }
- Destructuration : const { nom, age } = etudiant
- Spread : const copie = { ...original, champModifie: nvValeur }
- Object.keys(), Object.values(), Object.entries()
- Méthode dans un objet : { calcMoyenne() { return ... } }

Question 3 — Construire le bulletin en objet

💡 **Indice :** *Relis la destructuration, le spread et Object.entries() dans le Chap. 5.*

a) Destructure l'objet suivant pour extraire `nom` et `moyenne`, en renommant `moyenne` en `moy`. Donne aussi une valeur par défaut de `0` à `absences` si la propriété est absente :

```
const bulletin = { nom: 'Emma Biyong', moyenne: 17, mention: 'Très Bien' }
```

b) Écris une fonction `mettreAJourBulletin(bulletin, modifications)` qui retourne un NOUVEAU bulletin fusionné sans modifier l'original. Utilise le spread operator. Exemple : `mettreAJourBulletin(b, { mention: 'Bien', rang: 1 })` doit retourner un nouvel objet.

c) Utilise `Object.entries()` et une boucle `for...of` pour afficher dans la console chaque propriété du bulletin sous le format `clé : valeur`.

d) BONUS — Écris une fonction `genererRapport(etudiants)` qui retourne un objet `{ totalEtudiants, moyenneClasse, meilleurEleve, nbRecus }`. Utilise `.reduce()`, `.find()` et les concepts des Chap. 3–5.

EXERCICE 03 ★★★

Le Mini Blog Personnel

Contexte : *Tu développes le moteur de données d'un blog. Il doit gérer les articles, les tags, les recherches et les statistiques, en combinant tous les concepts du cours.*

📖 **Rappel de cours — Révision transversale — Tous chapitres**

- Closures (Chap. 3) : une fonction qui garde en mémoire son environnement de création
- Fonctions d'ordre supérieur (Chap. 3) : fonctions qui prennent/retournent des fonctions
- Chainer les méthodes de tableau (Chap. 4) : .filter().map().reduce()
- Object.fromEntries() avec .map() pour transformer un objet (Chap. 5)
- Méthodes string avancées (Chap. 6) : .normalize(), .replace(), .split()
- Set pour dédupliquer (Chap. 13, concept de base utilisable dès maintenant)

Question 1 — Système de recherche avec closure

💡 **Indice :** *Relis les closures et fonctions d'ordre supérieur dans le Chap. 3.*

Voici les données du blog :

```
const articles = [  { id: 1, titre: 'Débuter avec JavaScript', tags:
['js','débutant','web'],    vues: 1240, likes: 87,  publie: true,  date: '2024-01-15' },
{ id: 2, titre: 'Les Closures expliquées', tags: ['js','avancé'],    vues: 890,  likes:
120, publie: true,  date: '2024-02-10' },  { id: 3, titre: 'Tailwind CSS v4',
tags: ['css','design','web'],    vues: 340,  likes: 45,  publie: false, date:
'2024-03-01' },  { id: 4, titre: 'React pour les nuls',     tags: ['react','js','web'],
vues: 2100, likes: 230, publie: true,  date: '2024-03-20' },  { id: 5, titre: 'Prisma &
Supabase',       tags: ['backend','db','js'],    vues: 560,  likes: 67,  publie: true,
date: '2024-04-05' },]
```

a) Écris une fonction `creerFiltre(cle, valeur)` qui retourne une FONCTION de filtrage. Exemple d'utilisation :

```
const filtrePublie  = creerFiltre('publie', true)const filtrePrivé   =
creerFiltre('publie', false)articles.filter(filtrePublie)  // retourne les articles
publiésarticles.filter(filtrePrivé)   // retourne les articles non publiés
```

C'est une closure : la fonction retournée se souvient de `cle` et `valeur`.

b) Écris une fonction `rechercherParTag(tag)` qui retourne tous les articles (publiés uniquement) dont le tableau `tags` contient ce tag. Utilise `.filter()` et `.includes()`.

c) Écris une fonction `creerComparateur(cle, ordre = 'desc')` qui retourne une fonction de comparaison pour `.sort()`. Exemple : `articles.sort(creerComparateur('vues'))` trie par vues décroissantes.

Question 2 — Statistiques du blog

💡 **Indice :** *Combine les méthodes de tableaux chaînées (Chap. 4) et la manipulation d'objets (Chap. 5).*

a) Calcule en une seule expression (sans variable intermédiaire) : le nombre total de vues de tous les articles **publiés**. Utilise `.filter()` puis `.reduce()`.

b) Collecte tous les tags uniques de tous les articles en un seul tableau (sans doublons). Astuce : utilise `.flatMap()` pour aplatir les tableaux de tags, puis `[...new Set(...)]` pour dédupliquer.

c) Crée un objet `statsParTag` qui compte combien d'articles publiés ont chaque tag. Résultat attendu : `{ js: 4, web: 3, ... }`. Utilise `.filter()`, `.flatMap()` et `.reduce()`.

d) En utilisant `Object.entries()` et `.sort()`, trouve le tag le plus populaire (celui qui apparaît le plus dans les articles publiés).

Question 3 — Slugifier les titres

💡 **Indice :** *Relis les méthodes string avancées (Chap. 6) : .normalize(), .replace(), .toLowerCase().*

a) Écris une fonction `toSlug(titre)` qui transforme un titre d'article en URL-friendly slug. Étapes attendues :

```
'Débuter avec JavaScript'  →  'debuter-avec-javascript''Les Closures expliquées'
→  'les-closures-expliquees''React & Tailwind v4.0 !'  →  'react-tailwind-v4-0'
```

Utilise dans l'ordre : `.toLowerCase()`, `.normalize('NFD')` + `.replace()` pour les accents, puis `.replace()` pour les caractères non alphanumériques, et enfin `.replace()` pour nettoyer les tirets en début/fin.

b) Écris une fonction `truncate(texte, max = 100)` qui tronque un texte trop long en ajoutant '...' à la fin. Si le texte fait moins de `max` caractères, il est retourné intact. Le résultat tronqué doit faire exactement `max` caractères au total.

c) BONUS — Assemblage final Écris une fonction `formatArticle(article)` qui retourne un nouvel objet enrichi de : `slug` (via toSlug), `extrait` (truncate(titre, 60)), et `tagsStr` (les tags

joints par `' #'`, ex : `'#js #web #débutant'`). Combine les fonctions que tu as écrites dans les questions précédentes.